

POSITION BASED DYNAMICS[1]

MATTHIAS MÜLLER, BRUNO HEIDELBERGER, MARCUS HENNIX, JOHN
RATCLIFF

Utkarsh Mall & Nitesh Dudhey

08/04/2016

Objectives

An approach for physically based animation which work directly on the positions.



Objectives

An approach for physically based animation which work directly on the positions.



Target

- Stability, Robustness and Speed in animation.

Objectives

An approach for physically based animation which work directly on the positions.



Target

- Stability, Robustness and Speed in animation.
- Direct control over positions of objects or vertices of a mesh which is desired in computer games.

08/04/2016

POSITION BASED SIMULATION

- Represent a dynamic object by a set of N vertices and M constraints.

Notations

- Represent a dynamic object by a set of N vertices and M constraints.
- $\forall i \in [1, \dots, N]$ m_i , $\mathbf{x}_i$ and $\mathbf{v}_i$ are mass, position and velocity of vertex i .

Notations

- Represent a dynamic object by a set of N vertices and M constraints.
- $\forall i \in [1, \dots, N]$ m_i , $\mathbf{x}_i$ and $\mathbf{v}_i$ are mass, position and velocity of vertex i .
- A constraint $j \in [1, \dots, M]$ consists of
 - a cardinality n_j ,
 - a function $C_j : \mathbb{R}^{3n_j} \rightarrow \mathbb{R}$,
 - a set of indices $\{i_1, \dots, i_{n_j}\}$, $i_k \in [1, \dots, N]$,
 - a stiffness parameter $k_j \in [0 \dots 1]$ and
 - a type of either *equality* or *inequality*.

Notations

- Represent a dynamic object by a set of N vertices and M constraints.
- $\forall i \in [1, \dots, N]$ m_i , $\mathbf{x}_i$ and $\mathbf{v}_i$ are mass, position and velocity of vertex i .
- A constraint $j \in [1, \dots, M]$ consists of
 - a cardinality n_j ,
 - a function $C_j : \mathbb{R}^{3n_j} \rightarrow \mathbb{R}$,
 - a set of indices $\{i_1, \dots, i_{n_j}\}$, $i_k \in [1, \dots, N]$,
 - a stiffness parameter $k_j \in [0 \dots 1]$ and
 - a type of either *equality* or *inequality*.
- Constraint j with type *equality* is satisfied if $C_j(\mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_{n_j}}) = 0$
- Constraint j with type *inequality* is satisfied if $C_j(\mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_{n_j}}) \geq 0$
- The stiffness parameter defines the strength of the constraint.

Algorithm

```
(01) forall vertices  $i$ 
(02)   initialize  $\mathbf{x}_i = \mathbf{x}_i^0$ ,  $\mathbf{v}_i = \mathbf{v}_i^0$ ,  $w_i = 1/m_i$ 
(03) endfor
(04) loop
(05)   forall vertices  $i$  do  $\mathbf{v}_i \leftarrow \mathbf{v}_i + \Delta t w_i \mathbf{f}_{ext}(\mathbf{x}_i)$ 
(06)   dampVelocities( $\mathbf{v}_i, \dots, \mathbf{v}_N$ )
(07)   forall vertices  $i$  do  $\mathbf{p}_i \leftarrow \mathbf{x}_i + \Delta t \mathbf{v}_i$ 
(08)   forall vertices  $i$ 
(09)     generateCollisionConstraints( $\mathbf{x}_i \rightarrow \mathbf{p}_i$ )
(10)   endloop
(11) loop solverIteration times
(12)   projectConstraints( $C_1, \dots, C_{M+M_{coll}}, \mathbf{p}_1, \dots, \mathbf{p}_N$ )
(13) endloop
```

```
(14) forall vertices  $i$ 
(15)    $\mathbf{v}_i \leftarrow (\mathbf{p}_i - \mathbf{x}_i)/\Delta t$ 
(16)    $\mathbf{x}_i \leftarrow \mathbf{p}_i$ 
(17) endfor
(18) velocityUpdate( $\mathbf{v}_1, \dots, \mathbf{v}_N$ )
(19) endloop
```

Initialization

```
(01) forall vertices  $i$   
(02)   initialize  $\mathbf{x}_i = \mathbf{x}_i^0$ ,  $\mathbf{v}_i = \mathbf{v}_i^0$ ,  $w_i = 1/m_i$   
(03) endfor
```

- Initialize values of positions and velocities of each vertex.
- Initialize Weights to each vertex(lighter the vertex's mass, more is the effect of correction due to constraints).

Velocity updates

```
(04) loop  
(05)   forall vertices  $i$  do  $\mathbf{v}_i \leftarrow \mathbf{v}_i + \Delta t w_i \mathbf{f}_{ext}(\mathbf{x}_i)$   
(06)   dampVelocities( $\mathbf{v}_i, \dots, \mathbf{v}_N$ )
```

- line (4) marks the beginning of simulation loop.
- Some external forces such as gravity cannot be represented as positional constraints.
- Hence there effect on system is takes place due to line (5).
- Velocity can be damped if necessary using line(6).
- A Method of damping is presented later.

Position estimation

(07) **forall** vertices i **do** $\mathbf{p}_i \leftarrow \mathbf{x}_i + \Delta t \mathbf{v}_i$

- Positions are estimated using an explicit Euler step, similar to force/impulse based methods.
- Note that position extrapolation like this leads to overshooting problem, which is the case with force/impulse based.
- These positions estimates are needed to be modified before updates.

Collision constraints

```
(08) forall vertices  $i$   
(09)    $generateCollisionConstraints(\mathbf{x}_i \rightarrow \mathbf{p}_i)$   
(10) endloop
```

- line (9) generates M_{coll} new constraints for collisions.
- These constraints change after every time step(since they are dependent on positions of vertices).

Constraint Solving

```
(11) loop solverIteration times  
(12)   projectConstraints( $C_1, \dots, C_{M+M_{coll}}$ ,  $\mathbf{p}_1, \dots, \mathbf{p}_N$ )  
(13) endloop
```

- We loop for $\text{solverIterations}(n_s)$ times, each time modifying estimated positions($\mathbf{p}_i$) slightly.
- The constraint solver uses the M fixed constraints along with M_{coll} collision constraints to modify position estimates.

Updating Positions and velocities

```
(14) forall vertices  $i$   
(15)    $\mathbf{v}_i \leftarrow (\mathbf{p}_i - \mathbf{x}_i)/\Delta t$   
(16)    $\mathbf{x}_i \leftarrow \mathbf{p}_i$   
(17) endfor
```

- Positions of vertices are updated to optimized estimates.
- Velocities updates are in exact correspondence with **VERLET INTERGRATION**.
- Verlet method stores the velocity implicitly as the difference between the current and the last position.

Modifying velocities

(18) *velocityUpdate*($\mathbf{v}_1, \dots \mathbf{v}_N$)

- In this step the velocity of colliding vertices is modified according to friction and coefficient of restitution.

STABILITY

- The scheme is unconditionally stable because the integration steps do not extrapolate blindly.
- The only possible source of instability is the solver itself, which uses **NEWTON-RAPHSON METHOD** to solve for valid position.
- This stability depends on the constraint function itself rather than time-step.

08/04/2016

- Takes inputs $M + M_{coll}$ constraints and the estimates $\mathbf{p}_1, ..\mathbf{p}_N$ for new locations of the points.

Constraint Solver

- Takes inputs $M + M_{coll}$ constraints and the estimates $\mathbf{p}_1, \dots, \mathbf{p}_N$ for new locations of the points.
- Tries to modify the estimates such that they satisfy all the constraints.

Constraint Solver

- Takes inputs $M + M_{coll}$ constraints and the estimates $\mathbf{p}_1, \dots, \mathbf{p}_N$ for new locations of the points.
- Tries to modify the estimates such that they satisfy all the constraints.
- Uses non linear Gauss-Seidel iteration method to obtain the constraints.

- Takes inputs $M + M_{coll}$ constraints and the estimates $\mathbf{p}_1, ..\mathbf{p}_N$ for new locations of the points.
- Tries to modify the estimates such that they satisfy all the constraints.
- Uses non linear Gauss-Seidel iteration method to obtain the constraints.
- **Advantage:**
 - In contrast to a Jacobi-type iteration, modifications to point locations immediately get visible to the process

- Takes inputs $M + M_{coll}$ constraints and the estimates $\mathbf{p}_1, ..\mathbf{p}_N$ for new locations of the points.
- Tries to modify the estimates such that they satisfy all the constraints.
- Uses non linear Gauss-Seidel iteration method to obtain the constraints.
- **Advantage:**
 - In contrast to a Jacobi-type iteration, modifications to point locations immediately get visible to the process
 - Faster convergence compared to Jacobi-type iterations.

- Takes inputs $M + M_{coll}$ constraints and the estimates $\mathbf{p}_1, ..\mathbf{p}_N$ for new locations of the points.
- Tries to modify the estimates such that they satisfy all the constraints.
- Uses non linear Gauss-Seidel iteration method to obtain the constraints.
- **Advantage:**
 - In contrast to a Jacobi-type iteration, modifications to point locations immediately get visible to the process
 - Faster convergence compared to Jacobi-type iterations.
- **Limitations:**
 - Gauss-Seidel is order dependent.

- Takes inputs $M + M_{coll}$ constraints and the estimates $\mathbf{p}_1, ..\mathbf{p}_N$ for new locations of the points.
- Tries to modify the estimates such that they satisfy all the constraints.
- Uses non linear Gauss-Seidel iteration method to obtain the constraints.
- **Advantage:**
 - In contrast to a Jacobi-type iteration, modifications to point locations immediately get visible to the process
 - Faster convergence compared to Jacobi-type iterations.
- **Limitations:**
 - Gauss-Seidel is order dependent.
 - In over-constrained situations, process can lead to oscillations if order is not kept constant.

Internal Constraints

- For internal constraints the linear and angular momentum are conserved.

Internal Constraints

- For internal constraints the linear and angular momentum are conserved.
- Collisions and attachment constraints are allowed to have global effects on the objects.

Internal Constraints

- For internal constraints the linear and angular momentum are conserved.
- Collisions and attachment constraints are allowed to have global effects on the objects.
- Consider an internal constraint with constraint function C , cardinality n , on points $\mathbf{p}_1, \dots, \mathbf{p}_n$ and stiffness k . Let $\mathbf{p}$ be $[\mathbf{p}_1^T, \dots, \mathbf{p}_n^T]^T$

Internal Constraints

- For internal constraints the linear and angular momentum are conserved.
- Collisions and attachment constraints are allowed to have global effects on the objects.
- Consider an internal constraint with constraint function C , cardinality n , on points $\mathbf{p}_1, \dots, \mathbf{p}_n$ and stiffness k . Let $\mathbf{p}$ be $[\mathbf{p}_1^T, \dots, \mathbf{p}_n^T]^T$
- C is independent of rigid body modes, i.e. translation and rotation.

Internal Constraints

- For internal constraints the linear and angular momentum are conserved.
- Collisions and attachment constraints are allowed to have global effects on the objects.
- Consider an internal constraint with constraint function C , cardinality n , on points $\mathbf{p}_1, \dots, \mathbf{p}_n$ and stiffness k . Let $\mathbf{p}$ be $[\mathbf{p}_1^T, \dots, \mathbf{p}_n^T]^T$
- C is independent of rigid body modes, i.e. translation and rotation.
- Rotating or Translating the vertices does not change the value of C .
 - gradient $\nabla_{\mathbf{p}} C$ is perpendicular to rigid body modes.

Internal Constraints

- For internal constraints the linear and angular momentum are conserved.
- Collisions and attachment constraints are allowed to have global effects on the objects.
- Consider an internal constraint with constraint function C , cardinality n , on points $\mathbf{p}_1, \dots, \mathbf{p}_n$ and stiffness k . Let $\mathbf{p}$ be $[\mathbf{p}_1^T, \dots, \mathbf{p}_n^T]^T$
- C is independent of rigid body modes, i.e. translation and rotation.
- Rotating or Translating the vertices does not change the value of C .
 - gradient $\nabla_{\mathbf{p}}C$ is perpendicular to rigid body modes.
- Choose $\Delta\mathbf{p}$ along $\nabla_{\mathbf{p}}C$, both momenta are automatically conserved.

Internal Constraints

- For internal constraints the linear and angular momentum are conserved.
- Collisions and attachment constraints are allowed to have global effects on the objects.
- Consider an internal constraint with constraint function C , cardinality n , on points $\mathbf{p}_1, \dots, \mathbf{p}_n$ and stiffness k . Let $\mathbf{p}$ be $[\mathbf{p}_1^T, \dots, \mathbf{p}_n^T]^T$
- C is independent of rigid body modes, i.e. translation and rotation.
- Rotating or Translating the vertices does not change the value of C .
 - gradient $\nabla_{\mathbf{p}}C$ is perpendicular to rigid body modes.
- Choose $\Delta\mathbf{p}$ along $\nabla_{\mathbf{p}}C$, both momenta are automatically conserved.
- given $\mathbf{p}$ find correction $\Delta\mathbf{p}$ such that $C(\mathbf{p} + \Delta\mathbf{p}) = 0$

$$C(\mathbf{p} + \Delta\mathbf{p}) \approx C(\mathbf{p}) + \nabla_{\mathbf{p}}C(\mathbf{p}) \cdot \Delta\mathbf{p} = 0 \quad (1)$$

Internal Constraints

- For internal constraints the linear and angular momentum are conserved.
- Collisions and attachment constraints are allowed to have global effects on the objects.
- Consider an internal constraint with constraint function C , cardinality n , on points $\mathbf{p}_1, \dots, \mathbf{p}_n$ and stiffness k . Let $\mathbf{p}$ be $[\mathbf{p}_1^T, \dots, \mathbf{p}_n^T]^T$
- C is independent of rigid body modes, i.e. translation and rotation.
- Rotating or Translating the vertices does not change the value of C .
 - gradient $\nabla_{\mathbf{p}}C$ is perpendicular to rigid body modes.
- Choose $\Delta\mathbf{p}$ along $\nabla_{\mathbf{p}}C$, both momenta are automatically conserved.
- given $\mathbf{p}$ find correction $\Delta\mathbf{p}$ such that $C(\mathbf{p} + \Delta\mathbf{p}) = 0$

$$C(\mathbf{p} + \Delta\mathbf{p}) \approx C(\mathbf{p}) + \nabla_{\mathbf{p}}C(\mathbf{p}) \cdot \Delta\mathbf{p} = 0 \quad (1)$$

$$\Delta\mathbf{p} = \lambda \nabla_{\mathbf{p}}C(\mathbf{p}) \quad (2) \quad 13$$

Solving Eq. (1) and Eq. (2) we get.

$$\Delta p = -\frac{C(\mathbf{p})}{\|\nabla_{\mathbf{p}} C(\mathbf{p})\|^2} \nabla_{\mathbf{p}} C(\mathbf{p}) \quad (3)$$

Solving Eq. (1) and Eq. (2) we get.

$$\Delta p = -\frac{C(\mathbf{p})}{\|\nabla_{\mathbf{p}} C(\mathbf{p})\|^2} \nabla_{\mathbf{p}} C(\mathbf{p}) \quad (3)$$

Which is regular Newton-Raphson step for the iterative solution of the non-linear equation.

Internal Constraints

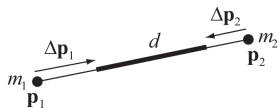
Solving Eq. (1) and Eq. (2) we get.

$$\Delta p = -\frac{C(\mathbf{p})}{\|\nabla_{\mathbf{p}} C(\mathbf{p})\|^2} \nabla_{\mathbf{p}} C(\mathbf{p}) \quad (3)$$

Which is regular Newton-Raphson step for the iterative solution of the non-linear equation.

If the points have individual masses then replace Eq 2 by

$$\Delta \mathbf{p}_i = \lambda w_i \nabla_{\mathbf{p}_i} C(\mathbf{p}) \quad (4)$$



Projection of the constraint
 $C(\mathbf{p}_1, \mathbf{p}_2) = |\mathbf{p}_1 - \mathbf{p}_2| - d$. The corrections $\delta \mathbf{p}_i$ are weighted according to the inverse masses $w_i = 1/m_i$

Internal Constraints

Solving Eq. (1) and Eq. (2) we get.

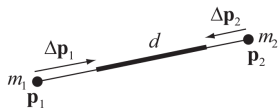
$$\Delta p = -\frac{C(\mathbf{p})}{\|\nabla_{\mathbf{p}} C(\mathbf{p})\|^2} \nabla_{\mathbf{p}} C(\mathbf{p}) \quad (3)$$

Which is regular Newton-Raphson step for the iterative solution of the non-linear equation.

If the points have individual masses then replace Eq 2 by

$$\Delta \mathbf{p}_i = \lambda w_i \nabla_{\mathbf{p}_i} C(\mathbf{p}) \quad (4)$$

- Project constraints if type is *equality* or *inequality* and $C(\mathbf{p}_1, \dots, \mathbf{p}_n) < 0$



Projection of the constraint
 $C(\mathbf{p}_1, \mathbf{p}_2) = |\mathbf{p}_1 - \mathbf{p}_2| - d$. The corrections $\delta \mathbf{p}_i$ are weighted according to the inverse masses $w_i = 1/m_i$

Internal Constraints

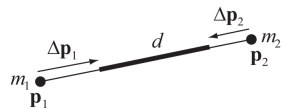
Solving Eq. (1) and Eq. (2) we get.

$$\Delta p = -\frac{C(\mathbf{p})}{\|\nabla_{\mathbf{p}} C(\mathbf{p})\|^2} \nabla_{\mathbf{p}} C(\mathbf{p}) \quad (3)$$

Which is regular Newton-Raphson step for the iterative solution of the non-linear equation.

If the points have individual masses then replace Eq 2 by

$$\Delta \mathbf{p}_i = \lambda w_i \nabla_{\mathbf{p}_i} C(\mathbf{p}) \quad (4)$$



Projection of the constraint
 $C(\mathbf{p}_1, \mathbf{p}_2) = |\mathbf{p}_1 - \mathbf{p}_2| - d$. The corrections
 $\delta \mathbf{p}_i$ are weighted according to the inverse
masses $w_i = 1/m_i$

- Project constraints if type is *equality* or *inequality* and $C(\mathbf{p}_1, \dots, \mathbf{p}_n) < 0$
- multiply the $\Delta \mathbf{p}$ by k to incorporate stiffness.
 - for n_s iterations replace k by $k' = 1 - (1 - k)^{1/n_s}$. Error $\Delta p(1 - k')^{n_s}$ becomes linear in k , i.e. error becomes independent of the number of iterations.

08/04/2016

For example consider the distance constraint function $C(\mathbf{p}_1, \mathbf{p}_2) = |\mathbf{p}_1 - \mathbf{p}_2| - d$.

The gradient are $\nabla_{\mathbf{p}_1} C(\mathbf{p}_1, \mathbf{p}_2) = \mathbf{n}$ and $\nabla_{\mathbf{p}_2} C(\mathbf{p}_1, \mathbf{p}_2) = -\mathbf{n}$, $\mathbf{n} = \frac{\mathbf{p}_1 - \mathbf{p}_2}{|\mathbf{p}_1 - \mathbf{p}_2|}$

For example consider the distance constraint function $C(\mathbf{p}_1, \mathbf{p}_2) = |\mathbf{p}_1 - \mathbf{p}_2| - d$.

The gradient are $\nabla_{\mathbf{p}_1} C(\mathbf{p}_1, \mathbf{p}_2) = \mathbf{n}$ and $\nabla_{\mathbf{p}_2} C(\mathbf{p}_1, \mathbf{p}_2) = -\mathbf{n}$, $\mathbf{n} = \frac{\mathbf{p}_1 - \mathbf{p}_2}{|\mathbf{p}_1 - \mathbf{p}_2|}$

The final corrections are:

$$\Delta \mathbf{p}_1 = -\frac{w_1}{w_1 + w_2} (|\mathbf{p}_1 - \mathbf{p}_2| - d) \frac{\mathbf{p}_1 - \mathbf{p}_2}{|\mathbf{p}_1 - \mathbf{p}_2|} \quad (5)$$

$$\Delta \mathbf{p}_2 = +\frac{w_2}{w_1 + w_2} (|\mathbf{p}_1 - \mathbf{p}_2| - d) \frac{\mathbf{p}_1 - \mathbf{p}_2}{|\mathbf{p}_1 - \mathbf{p}_2|} \quad (6)$$

- Collision constraints are generated every time step based on the current positions of the vertices.

Collision Detection

- Collision constraints are generated every time step based on the current positions of the vertices.
- For continuous collision handling test the ray $\mathbf{x}_i \rightarrow \mathbf{p}_i$ for each vertex.

Collision Detection

- Collision constraints are generated every time step based on the current positions of the vertices.
- For continuous collision handling test the ray $\mathbf{x}_i \rightarrow \mathbf{p}_i$ for each vertex.
- If ray enters an object
 - compute the entry point $\mathbf{q}_c$ and surface normal $\mathbf{n}_c$ at $\mathbf{q}_c$.

Collision Detection

- Collision constraints are generated every time step based on the current positions of the vertices.
- For continuous collision handling test the ray $\mathbf{x}_i \rightarrow \mathbf{p}_i$ for each vertex.
- If ray enters an object
 - compute the entry point $\mathbf{q}_c$ and surface normal $\mathbf{n}_c$ at $\mathbf{q}_c$.
 - Add *inequality* constraint with constraint function $C(\mathbf{p}) = (\mathbf{p} - \mathbf{q}_c) \cdot \mathbf{n}_c$ and stiffness $k = 1$.

Collision Detection

- Collision constraints are generated every time step based on the current positions of the vertices.
- For continuous collision handling test the ray $\mathbf{x}_i \rightarrow \mathbf{p}_i$ for each vertex.
- If ray enters an object
 - compute the entry point $\mathbf{q}_c$ and surface normal $\mathbf{n}_c$ at $\mathbf{q}_c$.
 - Add *inequality* constraint with constraint function $C(\mathbf{p}) = (\mathbf{p} - \mathbf{q}_c) \cdot \mathbf{n}_c$ and stiffness $k = 1$.
- If ray $\mathbf{x}_i \rightarrow \mathbf{p}_i$ completely lies inside an object, use static collision handling.
 - Compute surface point $\mathbf{q}_s$ which is closest to $\mathbf{p}_i$ and surface normal $\mathbf{n}_s$ at $\mathbf{q}_s$.

Collision Detection

- Collision constraints are generated every time step based on the current positions of the vertices.
- For continuous collision handling test the ray $\mathbf{x}_i \rightarrow \mathbf{p}_i$ for each vertex.
- If ray enters an object
 - compute the entry point $\mathbf{q}_c$ and surface normal $\mathbf{n}_c$ at $\mathbf{q}_c$.
 - Add *inequality* constraint with constraint function $C(\mathbf{p}) = (\mathbf{p} - \mathbf{q}_c) \cdot \mathbf{n}_c$ and stiffness $k = 1$.
- If ray $\mathbf{x}_i \rightarrow \mathbf{p}_i$ completely lies inside an object, use static collision handling.
 - Compute surface point $\mathbf{q}_s$ which is closest to $\mathbf{p}_i$ and surface normal $\mathbf{n}_s$ at $\mathbf{q}_s$.
 - Add *inequality* constraint with constraint function $C(\mathbf{p}) = (\mathbf{p} - \mathbf{q}_s) \cdot \mathbf{n}_s$ and stiffness $k = 1$.

- Handle friction and restitution by manipulating velocities in step (19) of the algorithm.

- Handle friction and restitution by manipulating velocities in step (19) of the algorithm.
- Velocity for vertex for which collision constraint has been generated is dampened, by friction, perpendicular to the collision normal and reflected, by restitution, in the direction of the collision normal.

Collision Detection

- Handle friction and restitution by manipulating velocities in step (19) of the algorithm.
- Velocity for vertex for which collision constraint has been generated is dampened, by friction, perpendicular to the collision normal and reflected, by restitution, in the direction of the collision normal.
- If both bodies are colliding dynamically then
 - if a point $\mathbf{q}$ of one object moves through a triangle $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$ of another object insert *inequality* constraint

Collision Detection

- Handle friction and restitution by manipulating velocities in step (19) of the algorithm.
- Velocity for vertex for which collision constraint has been generated is dampened, by friction, perpendicular to the collision normal and reflected, by restitution, in the direction of the collision normal.
- If both bodies are colliding dynamically then
 - if a point $\mathbf{q}$ of one object moves through a triangle $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$ of another object insert *inequality* constraint
 - $C(\mathbf{q}, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3) = \pm(\mathbf{q} - \mathbf{p}_1)[(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1)]$

- Handle friction and restitution by manipulating velocities in step (19) of the algorithm.
- Velocity for vertex for which collision constraint has been generated is dampened, by friction, perpendicular to the collision normal and reflected, by restitution, in the direction of the collision normal.
- If both bodies are colliding dynamically then
 - if a point $\mathbf{q}$ of one object moves through a triangle $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$ of another object insert *inequality* constraint
 - $C(\mathbf{q}, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3) = \pm(\mathbf{q} - \mathbf{p}_1)[(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1)]$
 - Since the constraint function is independent of rigid body modes, it will correctly conserve linear and angular momentum.

Damping Velocities

The velocities can be dampened before they are used for the prediction of the new positions.

- (1) $\mathbf{x}_{cm} = (\sum_i \mathbf{x}_i m_i) / \sum_i m_i$
- (2) $\mathbf{v}_{cm} = (\sum_i \mathbf{v}_i m_i) / \sum_i m_i$
- (3) $\mathbf{L} = \sum_i \mathbf{r}_i \times m_i \mathbf{v}_i$
- (4) $\mathbf{I} = \sum_i \tilde{\mathbf{r}}_i \tilde{\mathbf{r}}_i^T m_i$
- (5) $\omega = \mathbf{I}^{-1} \mathbf{L}$
- (6) **forall** vertices i
- (7) $\Delta \mathbf{v}_i = \mathbf{v}_{cm} + \omega \times \mathbf{r}_i - \mathbf{v}_i$
- (8) $\mathbf{v}_i \leftarrow \mathbf{v}_i + k_{damping} \Delta \mathbf{v}_i$
- (9) **endfor**

Damping Velocities

The velocities can be dampened before they are used for the prediction of the new positions.

(1) $\mathbf{x}_{cm} = (\sum_i \mathbf{x}_i m_i) / \sum_i m_i$

(2) $\mathbf{v}_{cm} = (\sum_i \mathbf{v}_i m_i) / \sum_i m_i$

(3) $\mathbf{L} = \sum_i \mathbf{r}_i \times m_i \mathbf{v}_i$

(4) $\mathbf{I} = \sum_i \tilde{\mathbf{r}}_i \tilde{\mathbf{r}}_i^T m_i$

(5) $\omega = \mathbf{I}^{-1} \mathbf{L}$

(6) **forall** vertices i

(7) $\Delta \mathbf{v}_i = \mathbf{v}_{cm} + \omega \times \mathbf{r}_i - \mathbf{v}_i$

(8) $\mathbf{v}_i \leftarrow \mathbf{v}_i + k_{damping} \Delta \mathbf{v}_i$

(9) **endfor**

- $\mathbf{r}_i = \mathbf{x}_i - \mathbf{x}_{cm}$
- $\tilde{\mathbf{r}}_i$ is the 3 by 3 matrix with property $\tilde{\mathbf{r}}_i \mathbf{v} = \mathbf{r}_i \times \mathbf{v}$
- $k_{damping}$ in $[0...1]$ is the damping coefficient

- With the position based approach, attaching vertices to objects is quite simple.

Attachments

- With the position based approach, attaching vertices to objects is quite simple.
- The position of the vertex is simply set to the static target position.

Attachments

- With the position based approach, attaching vertices to objects is quite simple.
- The position of the vertex is simply set to the static target position.
- To make sure other constraints containing this vertex do not move it, its inverse mass w_i is set to zero.



Attachment of cloth to the top rigid body is done this way. For bottom attachment two way interactions are required.

CLOTH SIMULATION

- A cloth is represented by arbitrary triangle meshes.

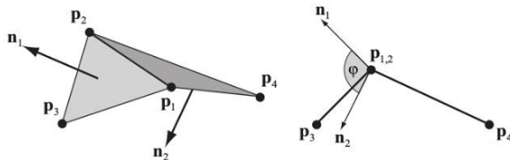
- A cloth is represented by arbitrary triangle meshes.
- Each edge is shared by at most 2 triangles.

Representation

- A cloth is represented by arbitrary triangle meshes.
- Each edge is shared by at most 2 triangles.
- Density ρ is given as mass per unit area kg/m^2 . Mass of a vertex is set to the sum of one third of the mass of each adjacent triangle.

- A cloth is represented by arbitrary triangle meshes.
- Each edge is shared by at most 2 triangles.
- Density ρ is given as mass per unit area kg/m^2 . Mass of a vertex is set to the sum of one third of the mass of each adjacent triangle.
- For each edge generate stretching constraint with:
 - $C_{stretch}(\mathbf{p}_1, \mathbf{p}_2) = |\mathbf{p}_1 - \mathbf{p}_2| - l_o$, l_o = initial length of the edge.
 - stiffness $k_{stretch}$ (global stretching constant of the cloth)
 - type *equality*

Representation

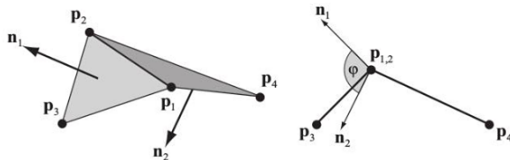


For bending resistance, the constraint function

$C(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4) = \arccos(\mathbf{n}_1 \cdot \mathbf{n}_2) - \varphi$ is used. The actual dihedral angle φ is measure as the angle between the normal of the two triangles.

- For each pair of each triangles $(\mathbf{p}_1, \mathbf{p}_3, \mathbf{p}_2)$ and $(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_4)$ generate constraints
 - $C(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4) = \arccos\left(\frac{(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1)}{\|(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1)\|} \cdot \frac{(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_4 - \mathbf{p}_1)}{\|(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_4 - \mathbf{p}_1)\|}\right) - \varphi_0$, φ_0 = initial dihedral angle between the two triangles.
 - k_{bend} (global bending stiffness of the cloth)
 - type *equality*

Representation



For bending resistance, the constraint function

$C(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4) = \arccos(\mathbf{n}_1 \cdot \mathbf{n}_2) - \varphi$ is used. The actual dihedral angle φ is measure as the angle between the normal of the two triangles.

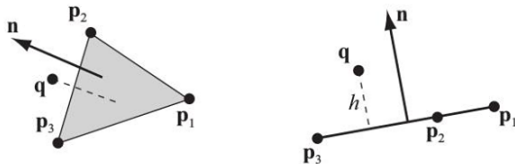
- For each pair of each triangles $(\mathbf{p}_1, \mathbf{p}_3, \mathbf{p}_2)$ and $(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_4)$ generate constraints
 - $C(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4) = \arccos\left(\frac{(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1)}{\|(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1)\|} \cdot \frac{(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_4 - \mathbf{p}_1)}{\|(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_4 - \mathbf{p}_1)\|}\right) - \varphi_0$, φ_0 = initial dihedral angle between the two triangles.

- k_{bend} (global bending stiffness of the cloth)
 - type *equality*

- The advantage of this bending term is that it is *independent of stretching*. 08/04/2016

- Collision detection and handling is done as explained before.
- To get two-way interactions, apply an impulse $m_i \Delta \mathbf{p}_i / \Delta t$ to rigid body at contact point.

Self Collision

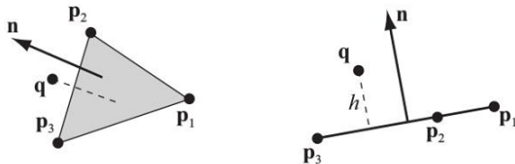


Constraint function $C(\mathbf{q}, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3) = (\mathbf{q}, \mathbf{p}_1) \cdot \mathbf{n} - h$ makes sure that $\mathbf{q}$ stays above the triangle $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$ by the cloth thickness h

- If a vertex $\mathbf{q}$ moves through a triangle $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$, we use the constraint function

- $C(\mathbf{q}, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3) = (\mathbf{q} - \mathbf{p}_1) \cdot \frac{(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1)}{|\mathbf{p}_2 - \mathbf{p}_1| \times |\mathbf{p}_3 - \mathbf{p}_1|} - h$

Self Collision



Constraint function $C(q, p_1, p_2, p_3) = (q, p_1) \cdot n - h$ makes sure that q stays above the triangle p_1, p_2, p_3 by the cloth thickness h

- If a vertex q moves through a triangle p_1, p_2, p_3 , we use the constraint function
 - $C(q, p_1, p_2, p_3) = (q - p_1) \cdot \frac{(p_2 - p_1) \times (p_3 - p_1)}{|p_2 - p_1| \times |p_3 - p_1|} - h$
- If the vertex enters from below with respect to the triangle normal
 - $C(q, p_1, p_2, p_3) = (q - p_1) \cdot \frac{(p_3 - p_1) \times (p_2 - p_1)}{|p_3 - p_1| \times |p_2 - p_1|} - h$
- h is the cloth thickness

Cloth Balloon



simulation of overpressure inside a character

- Model overpressure inside the closed triangle mesh.



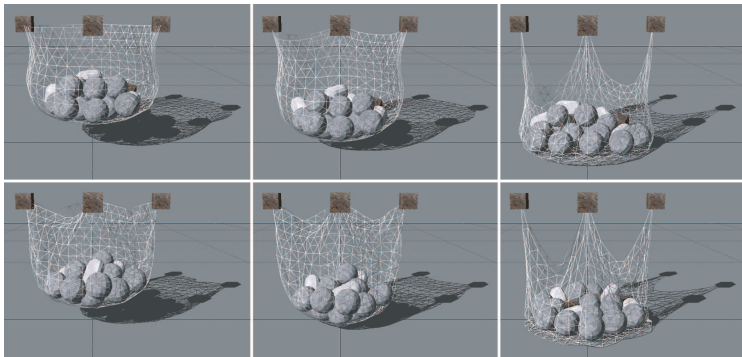
simulation of overpressure inside a character

- Model overpressure inside the closed triangle mesh.
- Add the constraint:
 - $C(\mathbf{p}_1, \dots, \mathbf{p}_N) = \left(\sum_{i=1}^{n_{triangles}} (\mathbf{p}_{t_1^i} \times \mathbf{p}_{t_2^i}) \cdot \mathbf{p}_{t_3^i} \right) - k_{pressure} V_0$
 - t_1^i, t_2^i and t_3^i are indices of the vertices of the triangle i .
 - The sum calculates the actual volume of the closed mesh.
 - V_0 is the original volume and $k_{pressure}$ overpressure factor.
 - stiffness $k = 1$
 - type *equality*

RESULTS

Results

- Independent Bending and Stretching stiffness provides more control over material properties compared to [2].

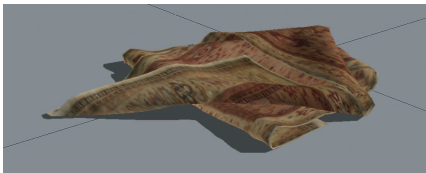


Bending and stretching are independent parameters. The top row shows $(k_{stretching}, k_{bending}) = (1,1)$, $(0.5,1)$ and $(0.01,1)$. The bottom row shows $(k_{stretching}, k_{bending}) = (1,0)$, $(0.5,0)$ and $(0.01,0)$

08/04/2016

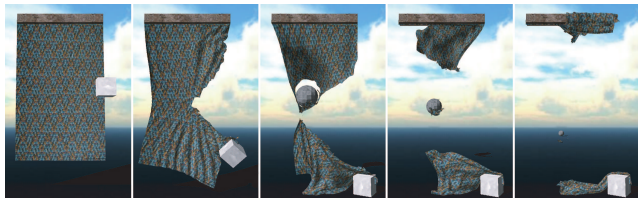
Results

- Real-Time self collision is possible efficiently.
- The piece of cloth shown, composed of 1364 vertices and 2562 triangles, simulation easily runs on 30 fps.
- The effect of friction is also shown in a rotating barrel.



Results

- Tearing of cloth is implemented efficiently using PBD.
- If the stretching of an edge exceeds some specified length, one of its vertex is divided in two equal masses and redistribute edges to these two vertices.
- piece of cloth consisting of 4264 vertices and 8262 triangles that is torn open by an attached cube and finally ripped apart by a thrown ball.
- This scene is simulated and rendered with 47 fps on average
- VIDEO



References

-  Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff.
Position based dynamics.
Journal of Visual Communication and Image Representation, 18(2):109 – 118, 2007.
-  Eitan Grinspun, Anil N. Hirani, Mathieu Desbrun, and Peter Schröder.
Discrete shells.
In *Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation, SCA '03*, pages 62–67, Aire-la-Ville, Switzerland, Switzerland, 2003. Eurographics Association.
-  Hendry Olsen.
sthlm Beamer Theme.